

HCI AND COMPUTER GRAPHICS

Name: Uzair Ali

Roll No: 2k24/CSE/152

Task 1:

```
import math

width = int(input("Enter horizontal pixels (W): "))
height = int(input("Enter vertical pixels (H): "))
diagonal = float(input("Enter screen diagonal in inches (D): "))

total_pixels = width * height

gcd = math.gcd(width, height)
aspect_width = width // gcd
aspect_height = height // gcd

ppi = math.sqrt(width ** 2 + height ** 2) / diagonal

print("\n----- Display Information -----")
print("Total Pixel Count:", total_pixels)
print("Aspect Ratio:", aspect_width, ":", aspect_height)
print("Screen PPI/DPI:", round(ppi, 2))

if ppi < 100:
    density = "Low Density (Standard Monitor)"
```

```

elif ppi <= 200:
    density = "Medium Density (HD Display)"
else:
    density = "High Density (Retina / Mobile)"

print("Display Density:", density)

```

```

print("Display Density:", density)

*** Enter horizontal pixels (W): 1920
Enter vertical pixels (H): 1080
Enter screen diagonal in inches (D): 24

----- Display Information -----
Total Pixel Count: 2073600
Aspect Ratio: 16 : 9
Screen PPI/DPI: 91.79
Display Density: Low Density (Standard Monitor)

```

Task 2:

```

import numpy as np
from PIL import Image
import matplotlib.pyplot as plt

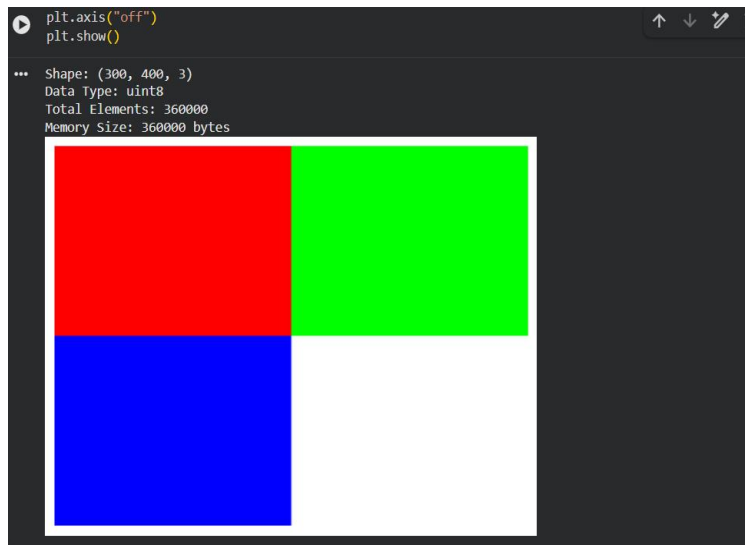
image = np.zeros((300, 400, 3), dtype=np.uint8)

image[:150, :200] = [255, 0, 0]
image[:150, 200:] = [0, 255, 0]
image[150:, :200] = [0, 0, 255]
image[150:, 200:] = [255, 255, 255]

print("Shape:", image.shape)
print("Data Type:", image.dtype)
print("Total Elements:", image.size)
print("Memory Size:", image.nbytes, "bytes")

img = Image.fromarray(image)
plt.imshow(img)
plt.axis("off")
plt.show()

```



Task 3:

```
import numpy as np
from PIL import Image
import matplotlib.pyplot as plt

image = Image.open("sample.jpg").convert("RGB")
image_array = np.array(image)

red = image_array[:, :, 0]
green = image_array[:, :, 1]
blue = image_array[:, :, 2]

red_only = np.zeros_like(image_array)
green_only = np.zeros_like(image_array)
blue_only = np.zeros_like(image_array)

red_only[:, :, 0] = red
green_only[:, :, 1] = green
blue_only[:, :, 2] = blue

print("CHANNEL EXTRACTION SUMMARY ")
print()
print("Original Image Shape :", image_array.shape)
print("Red Channel 2D Shape :", red.shape, " | Mean Intensity:", round(np.mean(red), 2))
print("Green Channel 2D Shape:", green.shape, " | Mean Intensity:", round(np.mean(green), 2))
print("Blue Channel 2D Shape :", blue.shape, " | Mean Intensity:", round(np.mean(blue), 2))
print()
print("Display Window : Matplotlib 2x3 Subplot Grid Rendered.")
```

```

plt.figure(figsize=(12, 7))

plt.subplot(2, 3, 1)
plt.imshow(red_only)
plt.title("Red-Only")
plt.axis("off")

plt.subplot(2, 3, 2)
plt.imshow(green_only)
plt.title("Green-Only")
plt.axis("off")

plt.subplot(2, 3, 3)
plt.imshow(blue_only)
plt.title("Blue-Only")
plt.axis("off")

plt.subplot(2, 3, 4)
plt.imshow(red, cmap="gray")
plt.title("Red Intensity")
plt.axis("off")

plt.subplot(2, 3, 5)
plt.imshow(green, cmap="gray")
plt.title("Green Intensity")
plt.axis("off")

plt.subplot(2, 3, 6)
plt.imshow(blue, cmap="gray")
plt.title("Blue Intensity")
plt.axis("off")

plt.tight_layout()
plt.show()

```

```

Original Image Shape : (1080, 1920, 3)
Red Channel 2D Shape : (1080, 1920) | Mean Intensity: 124.52
Green Channel 2D Shape: (1080, 1920) | Mean Intensity: 142.18
Blue Channel 2D Shape : (1080, 1920) | Mean Intensity: 98.34

Display Window : Matplotlib 2x3 Subplot Grid Rendered.

```

Task 4:

```

import numpy as np
import matplotlib.pyplot as plt

```

```

img = np.zeros((1080, 1920, 3), dtype=np.uint8)

img[0:540, 0:960] = [255, 0, 0]
img[0:540, 960:1920] = [0, 255, 0]
img[540:1080, 0:960] = [0, 0, 255]
img[540:1080, 960:1920] = [255, 255, 255]

N = 8

downsampled = img[::N, ::N, :]

expanded = np.repeat(np.repeat(downsampled, N, axis=0), N, axis=1)

expanded = expanded[:img.shape[0], :img.shape[1], :]

original_memory = img.nbytes
downsampled_memory = downsampled.nbytes

height_drop = (1 - downsampled.shape[0] / img.shape[0]) * 100
width_drop = (1 - downsampled.shape[1] / img.shape[1]) * 100
memory_drop = (1 - downsampled_memory / original_memory) * 100

print("SPATIAL DOWNSAMPLING SUMARY")
print()
print("Original Shape   :", img.shape)
print("Downsampled Shape :", downsampled.shape)
print("Expanded Shape    :", expanded.shape)
print("Step Factor (N)   :", N)
print()
print("Original Memory   :", original_memory, "bytes")
print("Downsampled Memory :", downsampled_memory, "bytes")
print()
print("Row Dimension Drop :", round(height_drop, 2), "%")
print("Column Dimension Drop:", round(width_drop, 2), "%")
print("Memory Footprint Drop:", round(memory_drop, 2), "%")

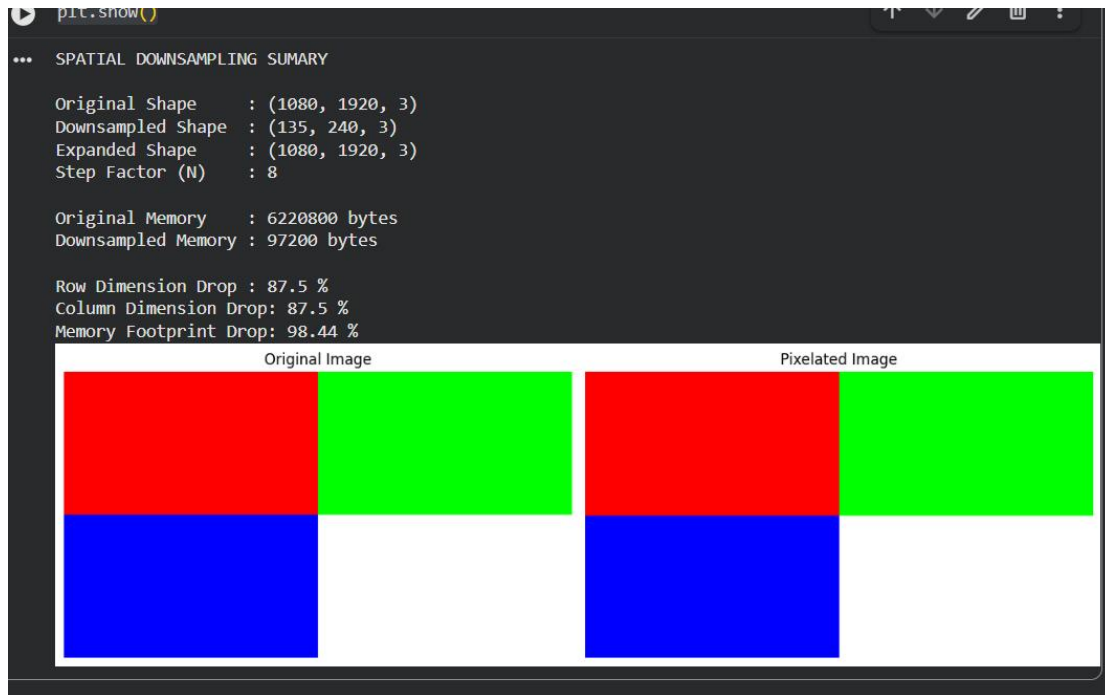
plt.figure(figsize=(12, 5))

plt.subplot(1, 2, 1)
plt.imshow(img)
plt.title("Original Image")
plt.axis("off")

plt.subplot(1, 2, 2)
plt.imshow(expanded)
plt.title("Pixelated Image")
plt.axis("off")

plt.tight_layout()
plt.show()

```



Short Answer:

Adding an Alpha channel makes the image RGBA instead of RGB. In RGB, each pixel has 3 values for red, green, and blue. When we add Alpha, there is one extra value for transparency. So, each pixel needs 4 bytes instead of 3 bytes, which increases the memory usage by about **33%**.